LARGE LANGUAGE MODELS: PRODUCTION AND DEPLOYMENT

From AI to Large Language Models

Module 1: Fine-Tuning LLMs — Part A: Foundations and Prerequisites

Lecture 1 · Foundation / Prerequisites · 60–90 minutes

TOPICS COVERED IN THIS MODULE

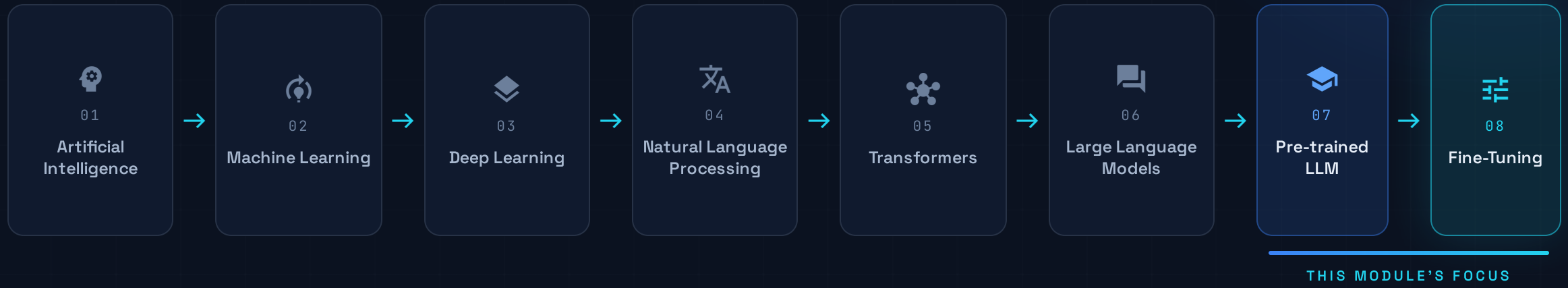
- Artificial Intelligence
- Machine Learning
- Deep Learning
- Natural Language Processing
- Tokens & Tokenization
- Embeddings
- Neural Networks
- Transformers
- Attention
- Large Language Models
- Training & Inference



MODULE 1 · PART A — ROADMAP

Where Does Fine-Tuning Fit?

Every slide in this lecture fills in one box of this chain — moving from broad AI down to fine-tuning.



Fine-tuning is the final step of this journey — and the **focus of this module**.

SECTION 01 / 04

PART A · FOUNDATIONS AND PREREQUISITES

Foundations: AI, Machine Learning & Neural Networks

> From intelligent machines to learned parameters — the vocabulary everything else builds on.

01 Foundations

02 From Text to Numbers

03 Attention & Transformers

04 LLMs, Training & Fine-Tuning

SECTION 01 — FOUNDATIONS

What Is Artificial Intelligence?

> **Artificial Intelligence** is the ability of a computer system to perform tasks that normally require human intelligence.

WHAT CAN AI DO?

- Understanding language
- Recognizing images
- Making decisions
- Solving problems
- Generating text
- Translating languages
- Recognizing speech

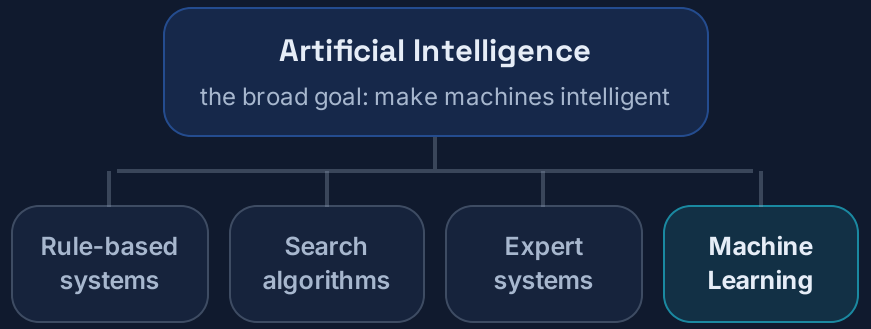
Where Do We See AI?

APPLICATION		AI CAPABILITY
Google Maps	→	Route prediction
Gmail	→	Spam detection
Netflix	→	Recommendations
Face Unlock	→	Face recognition
Alexa/Siri	→	Speech recognition
ChatGPT	→	Language generation
Banking	→	Fraud detection
Healthcare	→	Medical image analysis
Autonomous vehicles	→	Object detection

Each application on the right is an AI system performing a task that

SECTION 01 — FOUNDATIONS

AI vs Machine Learning



ML IS ONE BRANCH OF THE AI UMBRELLA

AI is the destination — **intelligent behavior**. Machine Learning is one way of getting there: learning the rules from data instead of writing them by hand.

TRADITIONAL PROGRAMMING

Data + Rules → Program → Answer

MACHINE LEARNING

Data + Correct Answers → Machine Learning → **Model**

↓ THEN LATER

New Data → Trained Model → **Prediction**

⚠ Rules like `IF "win money" → spam` break when spammers write 'You are selected for an exclusive financial reward.'

Traditional programs learn nothing. Machine-learning algorithms learn patterns from examples.

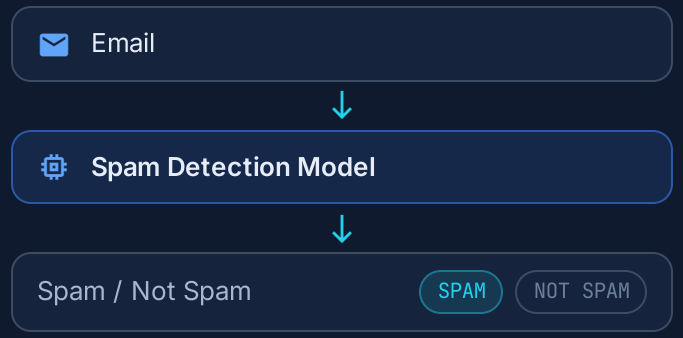
SECTION 01 — FOUNDATIONS

Models, Parameters & Training

$y = wx + b$

1 What is a Model?

A model is a **mathematical system** that has learned patterns from training data and can make predictions on new data.



2 Parameters

Parameters are values **learned automatically** during training — weights and biases.

$$y = wx + b$$

x = input · w = weight · b = bias · y = prediction

"This model has 7 billion parameters"

≈ 7,000,000,000

LEARNED VALUES

3 Training

Training is the process through which a model learns appropriate parameter values **from data**.

1. Training Data
2. Model Prediction
3. Compare with Correct Answer
4. Calculate Error
5. **Update Parameters**

↻ **Repeat**

SECTION 01 — FOUNDATIONS

Deep Learning & Neural Networks

Deep Learning is a branch of Machine Learning based on **multilayer artificial neural networks**.

Artificial Intelligence → Machine Learning → Deep Learning

TRADITIONAL ML	Linear Regression · Decision Trees · Random Forest · SVM
DEEP LEARNING EXCELS AT	Images · Speech · Text · Video

A MULTILAYER NETWORK



many hidden layers → **depth**

The Neuron

A neural network consists of interconnected computational units called **neurons**.

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Input × Weight — each input's importance → **Combine** — weighted sum + b → **Activation function** — produces the output

THE LINK TO FINE-TUNING

Fine-tuning eventually means adjusting some of these learned weights so that the network becomes better at our specific task.

SECTION 02 / 04 — PART A · FOUNDATIONS AND PREREQUISITES

From Text to Numbers: NLP, Tokens & Embeddings

Computers don't read words — language must become **vectors** before a network can learn from it.

TEXT → TOKENS → TOKEN IDS → VECTORS

SECTION 02 — FROM TEXT TO NUMBERS

What Is Natural Language Processing?

NLP is the field of AI concerned with enabling computers to **process, understand and generate** human language.

WHAT NLP SYSTEMS DO

- Sentiment analysis
- Translation
- Summarization
- Question answering
- Text classification
- Named entity recognition
- Chatbots
- Text generation

THREE MINI EXAMPLES

SENTIMENT	"The new phone is amazing." → **Positive** "Battery life is terrible." → **Negative**
TRANSLATION	English "How are you?" → Hindi **"आप कैसे हैं?"**
SUMMARIZATION	`1000`-word article → `100`-word summary

HOW NLP EVOLVED

Rule-Based NLP → Statistical NLP → Machine Learning → Deep Learning → RNN / LSTM → **Transformer (2017)** → Large Language Models

Computers Work With Numbers, Not Words

Tokens & Tokenization

A GPU cannot directly process “I love artificial intelligence.” — we need:

Text → Tokens → Token IDs → Numerical Representations → Neural Network

DEFINITION — WHAT IS A TOKEN?

A **token** is a unit of text processed by an LLM. A token may represent a complete word, part of a word, punctuation, or special characters.

- complete word
- part of a word
- punctuation
- special characters

TOKENIZATION FLOW

1. User Sentence — “AI is powerful”
2. Tokenizer
3. Tokens — ["AI", " is", " powerful"]
4. Token IDs — [4812, 318, 3665]

One token ≠ one word

‘unbelievable’ may split into subword units (depending on the tokenizer) — and punctuation can also become tokens.

un + believ + able

WORKED EXAMPLE “I love machine learning” → ["I" " love" " machine" " learnir

From Token IDs to Embeddings

Token IDs & Embeddings

WHY TOKENS MATTER IN PRODUCTION

Context length · Memory consumption · Computation · Inference latency · API cost · Training cost

short prompt → fewer tokens → less computation · very long prompt → more tokens → more computation

VOCABULARY & TOKEN IDS

AI	→	251
is	→	72
useful	→	940
.	→	13

“AI is useful.” → [251, 72, 940, 13]

Token IDs have no linguistic meaning — 251 is simply the index in the tokenizer’s vocabulary.

EMBEDDINGS

An **embedding** is a numerical vector representing a token in a continuous vector space.

King	→	[0.2, 0.8, -0.3, …]
Queen	→	[0.3, 0.7, -0.2, …]
Car	→	[-0.8, 0.1, 0.6, …]

Similar tokens land close together in vector space — ready for computation.

PIPELINE

Word → Token → Token ID → Embedding Vector

Token ID identifies the token — “dog” → 1278

Embedding represents it for computation — “dog” → [0.21, -0.42, 0.91, …, 0.17]

SECTION 03 / 04

Attention & Transformers

A mechanism for deciding, token by token, what deserves focus.

IN THIS SECTION

- Context problem
- Attention
- Self-Attention
- Query / Key / Value
- Transformer architecture

Why Attention? Words Change Meaning With Context

THE CONTEXT PROBLEM

I deposited money in the **bank**.

We sat on the **bank** of the river.

SAME WORD — DIFFERENT MEANINGS

The animal didn't cross the street because **it** was tired.

`'it' = animal` most likely

The animal didn't cross the street because **it** was flooded.

`'it' = street` meaning flipped by one word

Attention

Allows a model to determine **which other tokens are most relevant** when processing a particular token.



Self-Attention

Allows **every token in a sequence** to examine relationships with other tokens in the same sequence.



Each token asks: which other tokens matter for understanding me?

Query, Key & Value

How attention computes

Q **Query**
WHAT I SEARCH WITH

"What am I **looking for**?"

K **Key**
WHAT I CONTAIN

"What information do I **contain**?"

V **Value**
WHAT I CONTRIBUTE

"What information should I **contribute**?"

THE ATTENTION EQUATION

$$\text{Attention}(Q, K, V) = \text{softmax}\left(QK^T / \sqrt{d_k} \right) \cdot V$$

Do not worry about memorizing this equation today.

LIBRARY ANALOGY

QUERY
Your request: "I need a book about Machine Learning"

KEY
Each book's label on the shelf

VALUE
The actual book content you take away

The system compares your **Query** with the **Keys** and retrieves the relevant **Values**.

Q × K → *how relevant are other tokens?* → Softmax → Attention weights → Weighted combination of V

The Transformer

A **Transformer** is a deep-learning architecture built around attention mechanisms for processing sequences.

Input Text → Tokenizer → Token IDs → Embeddings → Transformer Layers → Output Probabilities → Next Token

MULTI-HEAD ATTENTION

- H1 grammatical relation
- H2 subject/object relation
- H3 positional relation
- H4 semantic relation

↓ combined representation

Heads can specialize in different patterns, learned during training.

POSITIONAL INFORMATION

"Dog bites man."

"Man bites dog."

Same words, different order — **completely different meaning**.

- positional embeddings
- rotary positional embeddings (RoPE)

Meaning depends on tokens **AND** their order.

INSIDE A TRANSFORMER BLOCK

Input
↓
Self-Attention
↓
Add + Normalize
↓
Feed-Forward Network
↓
Add + Normalize
↓
Output

Block 1 → Block 2 → Block 3 → … → Block N

*"this is where the **deep** in deep learning comes from."*

SECTION 04 / 04

Large Language Models, Training & Fine-Tuning

Bringing it together: how base models are built — and how they become specialized.

IN THIS SECTION

- LLM definition
- Generation
- Pre-training
- Training vs inference
- Loss
- Fine-tuning

What Is a Large Language Model?

LLM, defined

> An LLM is a **large neural language model**, generally based on **transformer architectures**, trained on **very large text datasets**.

BREAK THE NAME INTO THREE WORDS

Large = huge numbers of parameters, training examples, computation

Language = works primarily with language / token sequences

Model = a learned mathematical system

MODEL FAMILIES YOU WILL MEET

GPT · Llama · Gemma · Mistral · Qwen

CAPABILITIES

- Text generation
- Question answering
- Coding
- Summarization
- Translation
- Classification
- Information extraction
- Reasoning over prompts

All of these emerge from the same token-by-token prediction machinery.

NEXT-TOKEN PREDICTION

How Does an LLM Generate an Answer?

"Twinkle twinkle little..." → your brain predicts **star** from context.

INPUT PROMPT

"The capital of India is

NEXT-TOKEN PROBABILITIES

Delhi	0.92
Mumbai	0.03
Kolkata	0.02
Chennai	0.01
Others	0.02

Model selects → Delhi then repeats the process

AUTOREGRESSIVE GENERATION

Each predicted token is appended to the context — then the model predicts again.

Prompt
↓
Predict Token 1
↓
Prompt + Token 1
↓
Predict Token 2
↓
Prompt + Token 1 + Token 2
↓
Predict Token 3
↓
...

MYTH CHECK

Does an LLM search the internet every time? No.

BASIC LLM

Prompt → Model Parameters → Generated Response

EXTERNAL SEARCH

Requires additional systems / tools surrounding the model:

Search engines · RAG · Databases · APIs

PHASE 1 — PRE-TRAINING

What Is Pre-Training?

Pre-training is the initial large-scale training phase in which an LLM learns general language patterns from massive amounts of data.

PRE-TRAINING PIPELINE

01 Huge Text Dataset → 02 Tokenization → 03 LLM Training → 04 Learn Model Parameters → 05 **Base · Pre-trained Model**

TRAINING SIGNAL — WORKED EXAMPLE

"Machine learning is a branch of → `artificial` TARGET

Training text: "Machine learning is a branch of artificial intelligence." — the model is asked to predict successive tokens.

WHAT IT LEARNS

- syntax
- grammar
- factual associations
- semantic relations
- patterns in code
- reasoning-like structures

ANALOGY

School + College Education → General Knowledge

comparable to

Large General Dataset → Pre-Training → General LLM

THEN SPECIALIZE

Medical Specialization (Cardiology · Neurology · Dermatology)

analogous to Domain-specific Fine-Tuning

Base LLM → Fine-Tuning → **Specialized LLM**

TWO MODES OF OPERATION

Training vs Inference

TRAINING	INFERENCE
Model learns	Model is used
Parameters updated	Usually parameters fixed
Requires training dataset	Requires user input
Computationally expensive	Usually comparatively cheaper
Happens before deployment	Happens during usage

Training = studying before the examination

Inference = answering the question in the examination

FORWARD VS BACKWARD PASS

FORWARD PASS

Input → Model → Prediction

BACKWARD PASS

Prediction
↓
Calculate Loss
↓
Calculate Gradients
↓
Update Weights

Training requires Forward + Backward · Inference primarily requires Forward.

TRAINING — LLM EXAMPLE

millions → billions of examples → learn parameters

INFERENCE — LLM EXAMPLE

user asks “What is machine learning?” → trained LLM → response

THE OPTIMIZATION LOOP

How Models Improve: Loss, Gradient Descent & Learning Rate

Loss Function

A loss function tells us how far the model's prediction is from the desired output.

↓ Goal: Minimize Loss

EXAMPLE — CORRECT NEXT WORD: DELHI

BEFORE Mumbai 70% · Delhi 20%

→ loss is **high**

AFTER Delhi 92%

→ loss becomes **lower**

Gradient Descent

An optimization process used to update model parameters in a direction that reduces loss.

Current Weights
↓
Calculate Loss
↓
Calculate Gradient
↓
Update Weights
↓
Lower Loss

Descending a mountain in fog — examine the slope, move downhill, repeat until you reach a low point.

Learning Rate

Controls how large each parameter update is.

VERY SMALL → **slow learning**

the step is tiny — progress takes forever

VERY LARGE → **unstable training**

you may jump across the valley

SUITABLE → **efficient learning**

steady downhill steps reach the low point

WHY FINE-TUNE?

What Does a Base LLM Know — and What Is It Missing?

A pre-trained / base LLM has **broad capabilities** — but may not know:

- Organization-specific response style
- Specialized labels
- Internal terminology
- Required output format
- Specific domain behavior
- Particular task conventions

WORKED EXAMPLE — UNIVERSITY HELPDESK

USER

“My result is showing ABS although I attended the exam.”

GENERIC LLM OUTPUT

“You should contact the examination department.”

UNIVERSITY WANTS

Category: Examination
Subcategory: Result discrepancy
Priority: High
Escalation: Exam Cell

How can we make the model consistently follow this behavior? → Fine-Tuning

DEFINITION — FINE-TUNING

Fine-tuning means continuing the training of an already pre-trained model using a smaller, task-specific or domain-specific dataset.

THE PIPELINE



BASE VS FINE-TUNED

Fine-Tuning in Practice

BASE LLM

"Payment not reflected after 24 hours."

OUTPUT

"Please contact customer support."

FINE-TUNED UNIVERSITY MODEL

"Payment not reflected after 24 hours."

OUTPUT

Department: Accounts Issue: Fee payment reconciliation

Priority: Medium

Suggested Action: Submit transaction ID through the fee portal

The base LLM **already understands English** — we are not teaching it the meanings of payment, student, fee, transaction from scratch; we are teaching it **how our application should respond** to this type of request.

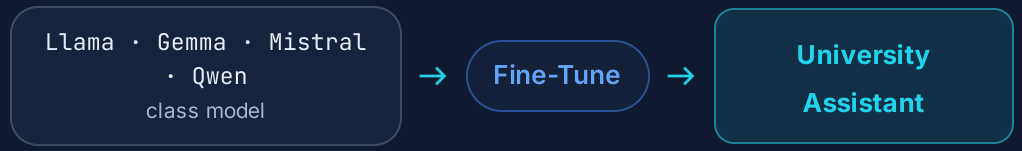
WHY NOT TRAIN AN LLM FROM SCRATCH?

- Huge datasets
- Massive GPU resources
- Long training time
- Distributed computing
- High financial cost
- Strong engineering expertise

THE PRACTICAL FORMULA

Fine-tuning = Existing Model + Smaller Dataset + Much Less Compute

THE PRAGMATIC PATH



CHECKPOINT

Quick Classroom Check

Ask students without showing answers first.

Q1 What does an LLM receive directly?

A. Human words
B. Tokens/token IDs
C. Images only
D. Database rows

ANSWER
B — Tokens/token IDs after preprocessing

Q2 What is a model parameter?

A. Batch size
B. Learning rate
D. Number of epochs

ANSWER
C — Learned weight

Q3 What mechanism allows transformers to model relationships between tokens?

ANSWER
Attention

Q4 What is inference?

ANSWER
Using a trained model to produce predictions/responses

Q5 Does a basic LLM automatically search Google for every answer?

ANSWER
No.

Q6 What is fine-tuning?

ANSWER
Further training an existing pre-trained model using task/domain-specific data to adapt its behavior

MODULE 1 · PART A — WRAP-UP

We Are Now Ready for Fine-Tuning

Foundation complete

THE MASTER CHAIN

13 concepts, one chain

AI → Machine Learning → Deep Learning →
Neural Networks → NLP → Tokens →
Embeddings → Attention → Transformers →
LLMs → Pre-training → Training vs Inference →
Fine-Tuning

ONE-MINUTE REVISION

USER TEXT
↓
TOKENIZER
↓
Token IDs
↓
EMBEDDINGS
↓
TRANSFORMER LAYERS
Attention · Feed Forward · Parameters
↓
NEXT-TOKEN SCORES
↓
SELECT NEXT TOKEN → ADD TO CONTEXT ↻ Repeat Generation
↓
FINAL RESPONSE

FROM BASE TO SPECIALIZED

BASE LLM
billions of parameters
↓
Task-Specific Dataset
↓
FINE-TUNING
some / all parameters adapted
↓
SPECIALIZED LLM
↓
INFERENCE

> If an LLM has billions of parameters, should we really update **all of them** every time? That sounds expensive.

NEXT LECTURE

Full Fine-Tuning → PEFT → LoRA → QLoRA